

CSCI 620: Project 1

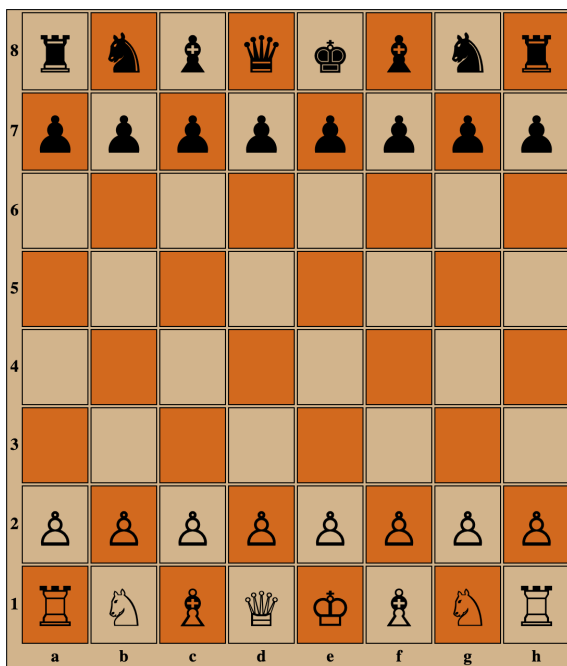
This is an individual assignment. Do your own work. Do not copy / paste anything from any outside source. You may brainstorm with others if needed. It is intended that you figure this all out on your own, but if you get stuck, and have put in a full effort to resolve your issues on your own, you may seek help during office hours. My help will be brief and high level, giving a hint to point you in the right direction.

Objectives

- Practice using HTML, CSS, Javascript, JQuery and Bootstrap to create a dynamically rendered HTML web page (single page load, no backend, page content rendered via JS DOM manipulation) containing a chess board with movable pieces using an HTML form.
- Compare and contrast Javascript's DOM (Document Object Model) manipulation using both vanilla javascript and the more streamlined JQuery syntax. To do this, you will implement two separate pages, one which dynamically renders the chessboard and moves the chess pieces using the “vanilla” javascript syntax, and then one that does the same thing using the more streamlined JQuery syntax.

Requirements

- Craft the two versions of a dynamically rendered HTML page which draws a standard 8 x 8 chessboard with all of the pieces in starting order, and each row and column labeled, as shown here:



- If you're unfamiliar with the game of chess, you should read this: https://en.wikipedia.org/wiki/Chess_piece
- Notes: The upper-left square is light colored, the black queen goes on a dark square, the white queen goes on a light square, the row labels start from 1 at the bottom and increase to 8 at the top, the column letters are displayed at the bottom of the chessboard and start with a on the left and increase to h on the right. Each chess piece is centered within its colored square. The labels are also centered.
- Use the background color “tan” for the light squares and the label areas. Use the background color “chocolate” for the dark squares.
- Each space on the board should be a square, and the board should be a square.

- Your CSS styles must be completely independent from your HTML, in other words, **do not use any HTML tag attributes to associate your CSS styles with your HTML tags**. (Use CSS **pseudoclass** and **child-combinator** selectors to create the alternating chessboard colors).
- Create a large heading above your chessboard containing “CSCI 620: Project 1 by “ followed by your full name as displayed in Canvas.
- Use the <hr> "horizontal rule" tag to create a divider line below your chessboard.
- Create a few paragraphs of text below your first horizontal rule, briefly describing the game of chess and its history (in your own words). You can do any online research needed for this, but do not copy/paste content from any outside source.
- Create a second horizontal rule and below the rules section, then add a bullet list after with a description of each type of chess piece by name (i.e. Pawn, Rook, Bishop, etc.). Describe the basic movement abilities of each piece. You can do any online research needed for this, but do not copy/paste content from any outside source.
- Create a third horizontal rule and below piece descriptions, create an About section with your full name as displayed in Canvas, a paragraph or two about yourself, your interests and goals, and a recent picture of yourself (using an tag) (you will create and add an image to your github repo).

Instructions / Hints

- Use the link provided in the Canvas assignment associated with this project to get access to github classroom and the starting repository for this project.
- Recommended: Use the provided github CodeSpaces as your development environment.
- You will create two separate versions of your dynamically generated website, one within the provided **v1_without_jquery** folder, and one within the **v2_with_jquery** folder.
- Begin by implementing and testing the **v1_without_jquery** version.
- Here are some CSS styles to place within chessboard.css to get you started:

```
body {
  padding: 40px;
}

table {
  margin: auto;
  border: 1px solid black;
}

td {
  background: tan;
  width: 60px;
  height: 60px;
  border: 1px solid black;
  font-size: 60px;
  text-align: center;
}
```

- Before tackling the steps of rendering the chessboard using Javascript and JQuery, you may want to first create a static version of the page so you can determine the exact page content to be generated.
- Use the standard set of HTML **table** tags to construct your chessboard grid within the body of your html (<table></table>, <tr></tr>, <td></td>, <th></th>, etc.). Hint: The “heading” table tags,

`<th></th>`, are useful for rendering the board labels. You may find this reference useful:

<https://developer.mozilla.org/en-US/docs/Learn/HTML/Tables/Basics>

- After the static version of your chessboard is rendering, it's time to tackle rendering it dynamically using vanilla javascript. To do this, you will remove all of the static HTML tags from the body section of your document, and replace it with just a placeholder pair of `<div></div>` tags. Add an id attribute tag with a unique value, and then add a new javascript function that gets called on page load to fill-in the contents inside this div tag (dynamically generate the chessboard content using vanilla javascript DOM manipulation). Be sure to separate the code that renders the board from the code that renders the chess pieces in their starting positions.

...continued on next page...

- The set of CSS Style Rules provided above, along with your HTML table tags, will give you a chessboard with nice borders, a tan background, and reasonable sizes for your chess pieces. You will need to figure out how to get the alternating dark / light background colors as shown in the image above (this is the hard part).

Hint: Use the **tr/td:nth-child(odd/even)** CSS "Combinator" Selectors, to change the background attribute for the selected **td** tags. The background of your chess table is light (tan), so you need to define the appropriate CSS selectors to designate the squares that should be dark (chocolate).

- Note: Row tags (<tr></tr>) and Cell tags within rows (<td></td>) are logically numbered (with respect to CSS Combinator Selectors) from left to right and top to bottom, starting with 1.
- Odd numbered cells within odd rows, and even numbered cells within even rows should have the CSS 'background:chocolate' property applied.

- Example CSS

```
/* make all even numbered cells have a blue background */
```

```
td:nth-child(even) {
    background: blue;
}
```

```
/* make all cells within odd numbered rows have a green background */
```

```
tr:nth-child(odd)>td {
    background: green;
}
```

```
/* make the odd numbered cells within even numbered rows have a red background */
```

```
tr:nth-child(even)>td:nth-child(odd) {
    background: red;
}
```

- The chess pieces are rendered inside of each table cell using the following HTML encoded Unicode characters, represented as "HTML Entities" (See <https://developer.mozilla.org/en-US/docs/Glossary/Entity>).

White King: ♔

Black King: ♚

White Queen: ♕

Black Queen: ♛

White Rook: ♖

Black Rook: ♜

White Bishop: ♗

Black Bishop: ♝

White Knight: ♘

Black Knight: ♞

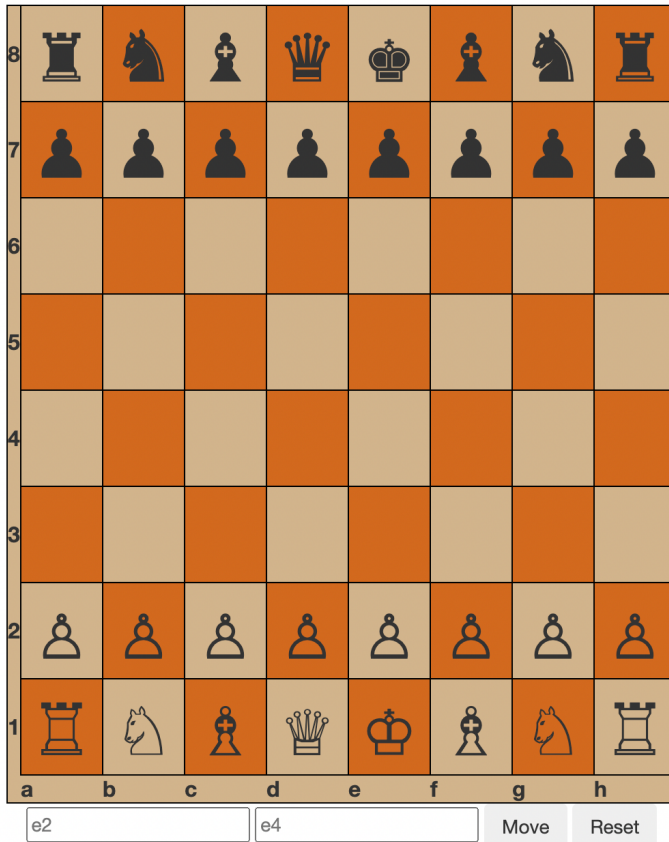
White Pawn: ♙

Black Pawn: ♟

- Use the following "non-breaking-space" code for empty spaces on the board:

- To test your web page, open the file chess.html in your development browser (Chrome is highly recommended for this as it has excellent developer tools, and also this is the browser I will be using to test and grade your submissions). Confirm that the chessboard renders correctly.

- After the **v1_without_jquery** chessboard is rendering correctly, it's time to add the chess piece movement feature.
- Add a set of controls just below your dynamic chessboard that allow chess piece movement using standard chess notation:



- By convention, chess moves are specified using the notation as shown above. For the starting position shown above, the move **e2 e4** means to advance the white pawn at square **e2** to square **e4**.
- Note: At this stage you do not need to implement chess move validation, just perform the moves designated by the user.
- Using a "**placeholder**" attribute on each **input** control, provide "hint" text as shown above ("e2" and "e4") for the expected input format.

<continued on next page...>

- If you haven't done so already, add **id** attributes to your `<td>` table elements, matching the standard chess notation for identifying each square on the board, "a1", "a2", etc. This will provide the needed linkage between your javascript logic and the chessboard HTML.
- Add **id** attributes to uniquely identify each of the **input** controls. Using **id="src"** for the first input control and **id="dst"** for the bottom input control will likely be convenient.
- Create two javascript functions within chessboard.js, one for moving the chess pieces attached to the **Move** button (via the **onclick** attribute), and one for resetting the board to the starting state attached to the **Reset** button (also via the **onclick** attribute).
- Implement the javascript function for **Reset**. Do not use **window.location.reload()** to reload the page, but leverage the javascript function to previously wrote to render the starting board position.
- Add Bootstrap styled **input** and **button** controls similar to what is shown the image above. They don't have to match the provided image exactly, just make sure the controls are rendered below the chessboard, are styled with bootstrap, and are usable and functional. Hint: Use the **onClick** HTML attribute to handle the **Move** and **Reset** actions.
- Implement the javascript function connected to the **Move** button. Start by implementing the javascript only version, using **document.getElementById(...).value** and **document.getElementById(...).innerHTML** to move the designated source piece to the designated destination square. Be sure to clear the source square after moving the piece.
- Verify that the **Move** and **Reset** buttons are working as expected using the vanilla Javascript syntax described above. Enter a valid chess move (e.g. "e2" "e4") and select the **Move** button. Confirm that the chess piece moves correctly. Then select the Reset button and confirm that the chessboard resets to its starting position.
- Now implement the JQuery version of the **Move** function, eliminating the use of **document.getElementById(...)**, and instead use the streamlined **JQuery** syntax.
- Verify that the **Move** and **Reset** buttons are working as expected using the JQuery DOM manipulation syntax. Enter a valid chess move (e.g. "e2" "e4") and select the **Move** button. Confirm that the chess piece moves correctly. Then select the Reset button and confirm that the chessboard resets to its starting position.
- Be sure to commit and push your final set of changes to the provided github repo before the cut-off date and time provided in Canvas.
- See the Canvas assignment for additional requirements (the Report that is also due).